

SoundScore V1

Mobile App Product + Architecture Blueprint (iOS + Android)

Prepared by Madhav Chauhan

March 5, 2026

Contents

- 1. Executive summary
- 2. Market intelligence and product strategy
- 3. Product definition and V1 success criteria
- 4. Mobile experience design (iOS/Android flows + screens)
- 5. Mobile app architecture (client-side deep dive)
- 6. System architecture overview (backend + providers)
- 7. Deep architecture (end-to-end subsystems)
- 8. API design (implementation-ready)
- 9. Infrastructure, DevOps, and operations
- 10. Security, privacy, and compliance
- 11. Roadmap and rollout plan (mobile-first)
- Appendices: event schemas + reference links

1. Executive summary

SoundScore is a mobile-first product I am designing as a "Letterboxd for music": a dedicated place to log albums I actually listened to, rate them on a consistent scale, write real reviews (optionally), build lists, and make taste legible on a profile - then layer friend-first social discovery on top.

V1 design realities I am architecting around

The surface area is crowded (ratings, reviews, lists, profiles). Winning comes from friction reduction, culture, and distribution loops - not just features.

Low-effort capture scales. V1 must make logging feel like a swipe/tap, not an essay prompt.

Spotify is not a stable scaling foundation (Dev Mode constraints). The core product must work without Spotify, with Spotify treated as an optional enhancement.

This report documents an end-to-end V1 that is:

- Mobile-first (iOS + Android are the primary clients; the app is the product).
- Provider-agnostic (SoundScore owns canonical IDs; external providers are adapters).
- Event-driven where it matters (listening events + activity events).
- Operationally sane (modular monolith + queue + background workers).
- Built for trust (export, delete, disconnect compliance as real product surfaces).

2. Market intelligence and product strategy

2.1 Competitive landscape (what SoundScore competes with)

I am competing with three product categories:

1. Mobile-first music journals (ratings, reviews, lists, identity).
2. Entrenched databases (huge catalog moats, web-first UX).
3. Passive social listening (automatic feeds generated from existing behavior).

2.2 Success patterns (what the best consumer products do)

- Identity becomes meaningful fast: a profile looks "real" within the first session.
- Capture is effortless: the system creates content from behavior; writing is optional.
- Social is scoped: friend-first avoids spam and keeps the product safe and positive.
- Share objects travel: recap cards, list cards, and profile cards drive acquisition.
- Trust compounds: exportability, reliability, and honest communication become differentiators.

2.3 Platform constraints I architect around

Because streaming platforms can change access rules quickly, I treat them as integrations - not foundations. SoundScore's core data model is independent of any provider, and providers are adapters behind a stable interface.

- Spotify Dev Mode limitations mean I cannot rely on it to scale user imports.
- Policy requirements demand clear disconnect + deletion and attribution/link-backs when provider content is displayed.
- Open-data options (MusicBrainz, ListenBrainz) de-risk the catalog and listening history.

3. Product definition and V1 success criteria

3.1 What SoundScore is

- Taste journal: log albums listened to, rate, review (optional), draft reviews, create lists.
- Taste identity: profile shows top albums, stats (genres/decades), and recency.
- Social discovery: follow friends, see activity, react/comment, explore album pages and lists.

3.2 The wedge (how V1 wins)

I am designing SoundScore to win on three axes:

1. Friction: one-tap logging from real listening context; reviews can be written later.
2. Culture: list/review norms that reward substance without demanding essays.
3. Distribution: shareable objects (weekly recap cards, list cards, profile cards).

3.3 What "successful V1" looks like

North Star outcomes

1. Taste identity becomes legible in under 2 minutes.
2. A weekly habit forms around logging + reacting + recaps.
3. Shares create acquisition and bring in friend clusters.
4. Trust is productized (export, deletion, privacy).

Concrete V1 targets (starter set)

Metric	Target	Why it matters
Time-to-first-log (median)	< 60 seconds	Captures the friction story; correlates strongly with D1 retention.
Feed load p95 (cached page 1)	< 400 ms	Mobile users abandon slow feeds quickly.
Profile load p95	< 300 ms	Profile is the identity surface that gets shared externally.
Shares / active user / week	Upward trend post-recaps	Signals product-market resonance and organic distribution.
Crash-free sessions	> 99.5%	Mobile reliability is brand trust; crashes kill retention.

4. Mobile experience design (iOS/Android flows + screens)

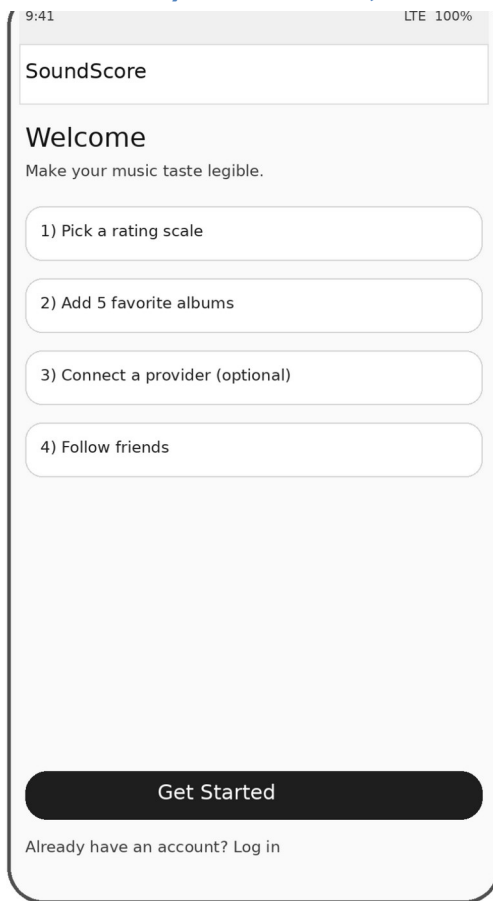
4.1 Mobile design principles I am enforcing

- Progressive disclosure: first session is not a writing assignment.
- Fast capture: logging is the primary call-to-action everywhere.
- Thumb-first ergonomics: key actions live in reachable zones; one-hand usage is respected.
- Safe social defaults: friend-first; public exposure is gradual.
- Shareability by design: every major surface can be turned into a share card.

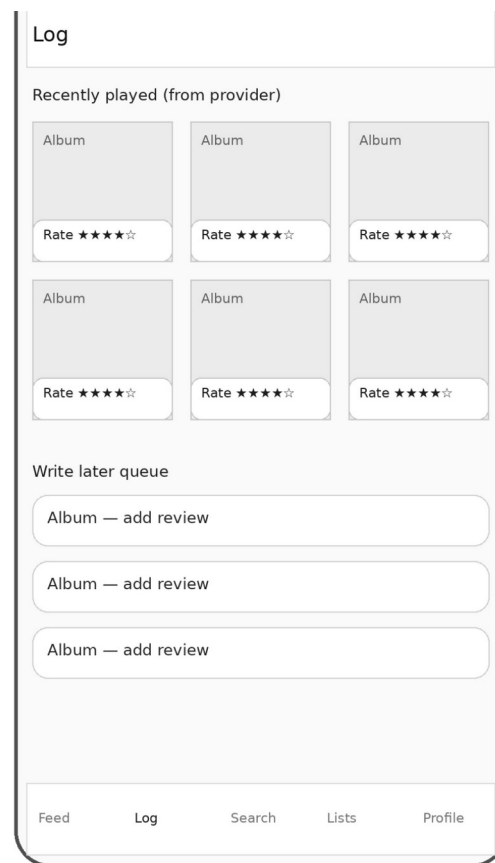
4.2 Core mobile journeys

1. Onboard -> identity (seed favorites, pick rating scale, optional provider connect).
2. Log -> rate -> (optional) review (write later queue).
3. Follow friends -> feed -> lightweight reactions (emoji + comments).
4. Create lists -> share list cards -> drive discovery.
5. Weekly recap card -> share -> acquisition loop.
6. Export / delete / disconnect -> trust and portability.

4.3 Low-fidelity wireframes (mobile-first)



Onboarding (fast identity)



Logging (recently played + 1-tap rate)

Album

Cover

Album Title

Artist • 2026

Rate

Review

Add to list

Listen

Friends

@friend ★★★★★☆ Short review...

@friend ★★★★★☆ Short review...

Community

Top review excerpt...

Top review excerpt...

Top review excerpt...

Album detail (friends + community)

Profile

@madhav

Taste Journal • 142 logs • 38 reviews

Top albums

Album

Album

Album

Album

Album

Album

Taste stats

Genres: Indie • Rap

Decades: 2010s

Avg rating: 3.9

Share profile card

Export data

Profile (taste artifact + share)

5. Mobile app architecture (client-side deep dive)

5.1 Framework decision (React Native + native escape hatches)

For V1, I am optimizing for speed of iteration while still building a production-grade mobile app. React Native (with Expo for build tooling) is the default choice because it allows one product team to ship iOS and Android simultaneously. Where platform depth matters, I plan native modules (Swift/Objective-C for iOS, Kotlin/Java for Android) behind stable JS interfaces.

When I drop to native code (examples)

OAuth flows and browser sessions (ASWebAuthenticationSession, Chrome Custom Tabs).

Share sheet customization, faster image rendering, and deep link handling.

Background tasks with platform-specific schedulers (BGTaskScheduler / WorkManager).

Home screen widgets (WidgetKit / Android widgets) if I ship them post-V1.

5.2 Mobile app layering model

Mobile App Architecture (iOS + Android)

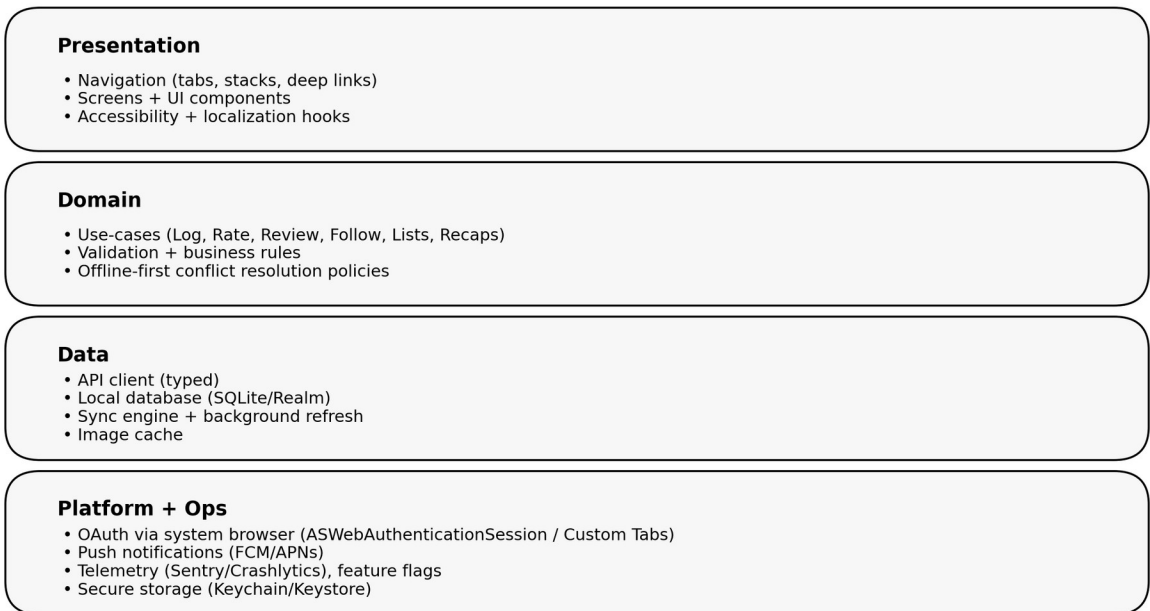


Figure 1. Client-side architecture layers (presentation -> domain -> data -> platform).

5.3 State management and data flow

- Server-state: TanStack Query (or equivalent) for caching, retries, and background refetch policies.
- UI state: lightweight store (Zustand/Redux Toolkit) for navigation/UI-only state, drafts, and transient flags.
- Typed APIs: OpenAPI-generated clients (preferred) to keep mobile + backend contracts aligned.
- Optimistic updates for reactions/ratings, backed by idempotency keys to avoid duplicates.

5.4 Offline-first strategy (practical V1)

Mobile networks are unreliable; I treat offline as normal. V1 uses an offline-first approach for the highest-frequency actions: viewing feed/profile, browsing cached albums, writing review drafts, and queueing writes.

- Local DB: SQLite (via WatermelonDB) or Realm for structured caching of feed items, profiles, lists, drafts.
- Outbox pattern: every write creates an outbox record with an idempotency key; the sync engine flushes when online.
- Conflict policy: last-write-wins for reactions; review edits use revision IDs; ratings are upserts keyed by (user, album).
- Background refresh: constrained; use push notifications for recap-ready and social events rather than polling.

5.5 Navigation, deep links, and share flows

- Navigation: bottom tabs (Feed, Log, Search, Lists, Profile) with stack navigators per tab.
- Deep links: universal links (iOS) + app links (Android) for albums, lists, profiles, and recaps.
- Share cards: generated server-side for consistent typography; cached via CDN; share sheet uses a single image + deep link URL.
- Provider Listen buttons: open external provider apps using safe deep links; fall back to web if app not installed.

5.6 Push notifications

Push is core to retention on mobile. V1 uses FCM as the unified send layer, with APNs for iOS delivery.

- Notification types: new follower, comment/reaction on my activity, recap ready, write-later gentle nudge.
- Dedup: collapse keys prevent spammy bursts (especially for reactions).
- User controls: per-type toggles + quiet hours.
- Safety: never include sensitive content in push payloads; fetch details in-app after open.

5.7 Mobile performance budgets

Surface

Budget (target)

How I enforce it

Cold start	< 2.0s to first screen	Minimize JS bundle; defer non-critical init; preload fonts/images carefully.
Feed scroll	60fps sustained	Virtualized lists; memoized cells; image caching; avoid overdraw.
Image loading	< 200ms cache hit	Disk + memory cache; CDN thumbnails; prefetch on Wi-Fi.
Write actions	< 150ms UI response	Optimistic updates; outbox sync in background.

6. System architecture overview (backend + providers)

Even though SoundScore is a mobile product, the backend architecture is what makes the mobile experience feel instant: fast feeds, reliable imports, durable drafts, and share cards that always load.

SoundScore V1 (Mobile-first) - High-level System Architecture

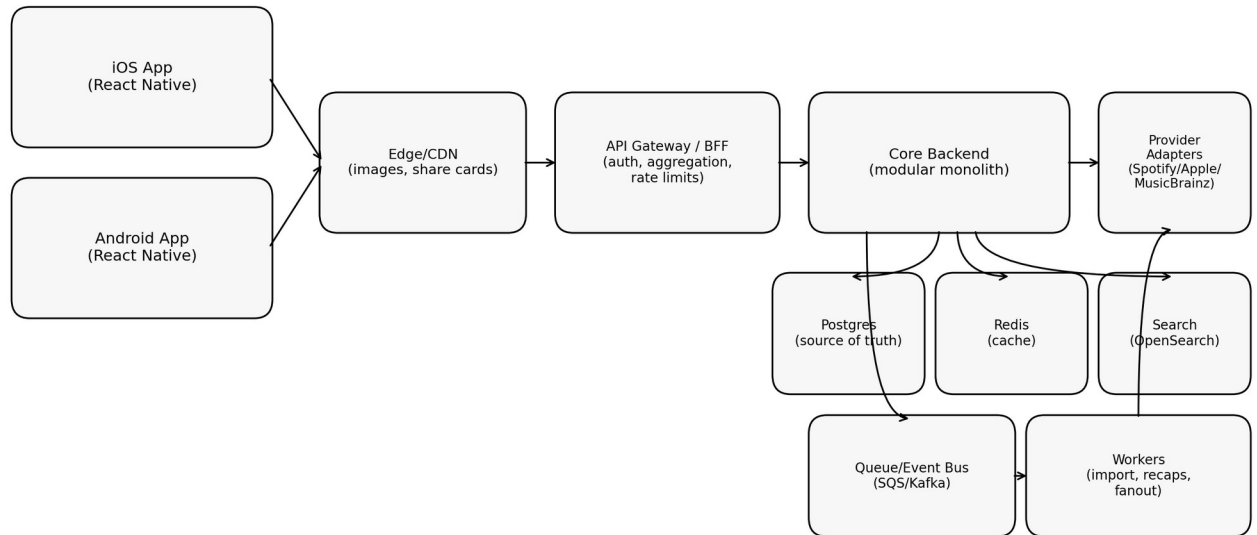


Figure 2. Mobile-first system architecture (iOS/Android clients as primary consumers).

6.1 Architecture philosophy

- Ship V1 fast without painting myself into a corner.
- Keep providers behind adapters so the product survives API policy shifts.
- Use a modular monolith with strict domain boundaries; add services only when thresholds force it.

6.2 Domain module map (backend)

- Identity & Auth
- Catalog & Mapping
- Listening Import
- Ratings & Reviews
- Social Graph
- Activity Feed
- Lists
- Recaps & Share Cards
- Notifications
- Moderation & Safety

- Trust Stack (export, delete, disconnect)

7. Deep architecture (end-to-end subsystems)

7.1 Data flow: import -> log -> social

The core loop is simple: listening context becomes listening events; listening events become logs and ratings; logs generate activity events; activity events form the feed and notifications.

1. Fetch provider listens (polling; later add webhook-like approximations if available).
2. Normalize into a listening_event schema (append-only).
3. Resolve catalog mapping (provider IDs -> SoundScore canonical album IDs).
4. Deduplicate and compress (avoid spam when someone loops an album).
5. Persist, cache, and emit social activity when the user rates/reviews.

7.2 Provider adapter architecture (Spotify/Apple/MusicBrainz)

I treat each external provider as an adapter behind a stable internal interface. This makes multi-provider support incremental rather than a rewrite.

```
interface ProviderAdapter {
    provider_name(): string

    // Identity
    exchange_oauth_code(code): TokenBundle
    refresh_token(refresh_token): TokenBundle
    revoke(token): void

    // Listening
    fetch_recent_plays(user_provider_identity, since_ts, limit): RawListen[]

    // Catalog
    lookup_album(provider_album_id): RawAlbum
    search_album(query): RawAlbum[]
}
```

7.3 Catalog and ID mapping (provider independence)

- SoundScore owns canonical entities: Artist, Album (release), Track (recording).
- A mapping table connects canonical IDs to provider IDs with confidence and provenance.

Practical V1 mapping algorithm

1. If provider includes a stable external ID already mapped -> use it.
2. Else attempt deterministic match using normalized artist/title, release-year window (+/-1), and track count.

3. If confidence $\geq$ threshold -> create mapping; else mark pending and request user confirmation only when needed.

7.4 Listening events as ground truth

- listening_event is append-only: what was played, when, and from which source.
- ratings and reviews attach to albums and can reference a listening event.
- Dedup key: (user_id, album_id, floor(played_at to 10 minutes)); store count to avoid feed flooding.

7.5 Activity feed (ranking + scaling)

V1 starts with fanout-on-read and a strong cache story:

```
SELECT * FROM activity_event
WHERE actor_id IN (
  SELECT followee_id FROM follow_edge WHERE follower_id = :me
)
AND created_at < :cursor
ORDER BY created_at DESC
LIMIT :limit;
```

- Cache feed page 1 per user for 30-120 seconds in Redis.
- Ranking heuristics (V1): boost reviews over ratings, boost mutuals, downrank repetitive events.
- When graphs grow (median follows > ~200 or p95 breaks), move to fanout-on-write with feed inbox tables.

7.6 Lists as distribution

- Lists are first-class surfaces: fast creation, ordered items, notes, and share card generation.
- Lists create internal discovery (trending lists) and external acquisition (share cards).

7.7 Recaps and share card generation

- Weekly job computes per-user aggregates (top albums, rating distribution, taste deltas, streaks).
- Store recap JSON and render share cards server-side for consistent visual identity.
- Share cards are served via CDN and opened via universal/app links back into the app.

7.8 Trust stack (export, deletion, portability)

- Export: snapshot bundle (ratings, reviews, lists, follows, listening history where allowed).
- Deletion: background job that removes or anonymizes content per policy.
- Provider disconnect: revoke tokens and cease processing provider personal data.

8. API design (implementation-ready)

8.1 Conventions

- Cursor-based pagination for feeds and comments.
- Idempotency keys for all writes (critical for mobile retries/offline outbox).
- Consistent error envelope; typed error codes for client UX.
- Versioned endpoints (/v1/...).

8.2 Endpoint set (starter)

Area	Endpoints
Auth	POST /v1/auth/signup POST /v1/auth/login POST /v1/auth/refresh
Providers	POST /v1/providers/:provider/connect POST /v1/providers/:provider/disconnect
Catalog	GET /v1/search?q=... GET /v1/albums/:id
Log & opinions	GET /v1/log/recently-played POST /v1/ratings POST /v1/reviews PUT /v1/reviews/:id
Social	POST /v1/follow/:user_id DELETE /v1/follow/:user_id GET /v1/feed POST /v1/activity/:id/react POST /v1/activity/:id/comment
Lists	POST /v1/lists POST /v1/lists/:id/items GET /v1/lists/:id
Recaps	GET /v1/recaps/weekly/latest GET /v1/share/profile/:handle
Trust	POST /v1/account/export DELETE /v1/account

9. Infrastructure, DevOps, and operations

9.1 Recommended stack

- Mobile: React Native + Expo (EAS Build/Submit), with native modules where needed.
- Backend: TypeScript (Fastify/NestJS) or Go; modular monolith.
- Data: Postgres + Redis + OpenSearch/Meilisearch.
- Queue: SQS for V1; evolve later if necessary.
- Storage/CDN: S3 + CloudFront (share cards, thumbnails).
- Observability: OpenTelemetry + Sentry/Crashlytics + dashboards for p95 latency and queue lag.

9.2 Mobile CI/CD (iOS + Android)

- Pull request builds: EAS preview builds for QA on real devices.
- Release trains: TestFlight (iOS) + Internal Testing track (Android).
- Phased rollout: staged percentage rollouts to catch regressions early.
- OTA updates: used carefully (UI fixes, not auth/security flows) to stay within app store policies.

9.3 Testing strategy

- Unit: domain use-cases, mapping logic, reducers/stores, utility functions.
- Integration: API contract tests (OpenAPI snapshots) + end-to-end flows in staging.
- UI: smoke tests for onboarding/logging/feed using Detox/Appium.
- Observability as testing: crash-free sessions, ANR rates, and network error rates monitored continuously.

10. Security, privacy, and compliance

10.1 Threat model (mobile-specific)

- Account takeover (credential stuffing, session theft).
- Token leakage (provider tokens, refresh tokens).
- Abuse/spam (fake accounts, automated reactions/comments).
- Scraping (public content extraction at scale).
- Mobile reverse engineering (tampering, API misuse).

10.2 Controls

- Secure storage: Keychain/Keystore for refresh tokens; no tokens in plain AsyncStorage.
- Network security: TLS everywhere; rotate refresh tokens; short-lived access tokens.
- Rate limits + device fingerprinting (privacy-respecting) to stop automation.
- Report/block flows in-app; moderation tooling on backend.
- Audit logs for account actions; anomaly alerts for unusual sign-in patterns.

10.3 App store privacy posture

- Only request permissions when needed (contacts optional; notifications opt-in; no background tracking).
- Clear privacy policy and in-app disclosure for connected providers.
- Data export + deletion are user-accessible; provider disconnect is one tap.

11. Roadmap and rollout plan (mobile-first)

11.1 Beta phases

1. Provider-free beta: manual logging + social + lists + recaps (no external dependencies).
2. Provider-limited beta: Spotify allowlist cohort; iterate on import reliability.
3. Multi-provider beta: Apple Music and/or ListenBrainz adapters; improve mapping confidence.

11.2 Launch sequence

- Launch with share mechanics + export to establish trust.
- Grow through list culture and recaps; tune notifications carefully.
- Scale feed and ingestion as graphs grow; introduce fanout-on-write only when required.

Appendix A: Event schemas

Activity event

```
{
  "id": "01J...",
  "actor_id": "01J...",
  "type": "RATED_ALBUM" | "WROTE_REVIEW" | "CREATED_LIST" | "ADDED_LIST_ITEM",
  "object": { "type": "album" | "review" | "list", "id": "01J..." },
  "created_at": "2026-03-05T12:34:56Z",
  "payload": { "album_id": "01J...", "rating": 4.5, "review_id": null }
}
```

Listening event

```
{
  "id": "01J...",
  "user_id": "01J...",
  "album_id": "01J...",
  "played_at": "2026-03-05T12:10:00Z",
  "source": "spotify" | "apple" | "manual",
  "source_ref": { "provider_item_id": "...", "raw": {} }
}
```

Appendix B: Reference links

- Spotify Developer Policy: <https://developer.spotify.com/policy>
- Spotify Dev Mode update (Feb 2026): <https://developer.spotify.com/blog/2026-02-06-update-on-developer-access-and-platform-security>
- Spotify migration guide:
<https://developer.spotify.com/documentation/web-api/tutorials/february-2026-migration-guide>
- Spotify quota modes:
<https://developer.spotify.com/documentation/web-api/concepts/quota-modes>
- MusicBrainz data license (CC0 core): https://musicbrainz.org/doc/About/Data_License
- ListenBrainz docs: <https://listenbrainz.readthedocs.io/en/latest/index.html>